

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



Searching

Oleh:

Amalia Soraya

NIM. 2510817120002

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA

MODUL 5

Laporan Praktikum Algoritma & Struktur Data Modul 5 : Searching ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Amalia Soraya
NIM : 2510817120002

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
LINEAR SEARCH.....	5
A. Source Code.....	5
B. Output Program	6
C. Pembahasan	6
BINARY SEARCH	8
A. Source Code.....	8
B. Output Program	9
C. Pembahasan	9
ANALISIS PERFORMA	11
TAUTAN GIT	12

DAFTAR GAMBAR

Gambar 1. Screenshot Output Linear Search	6
Gambar 2. Screenshot Output Binary Search	9

DAFTAR TABEL

Tabel 1. Source Code Linear Search	5
Tabel 2. Source Code Binary Search.....	8

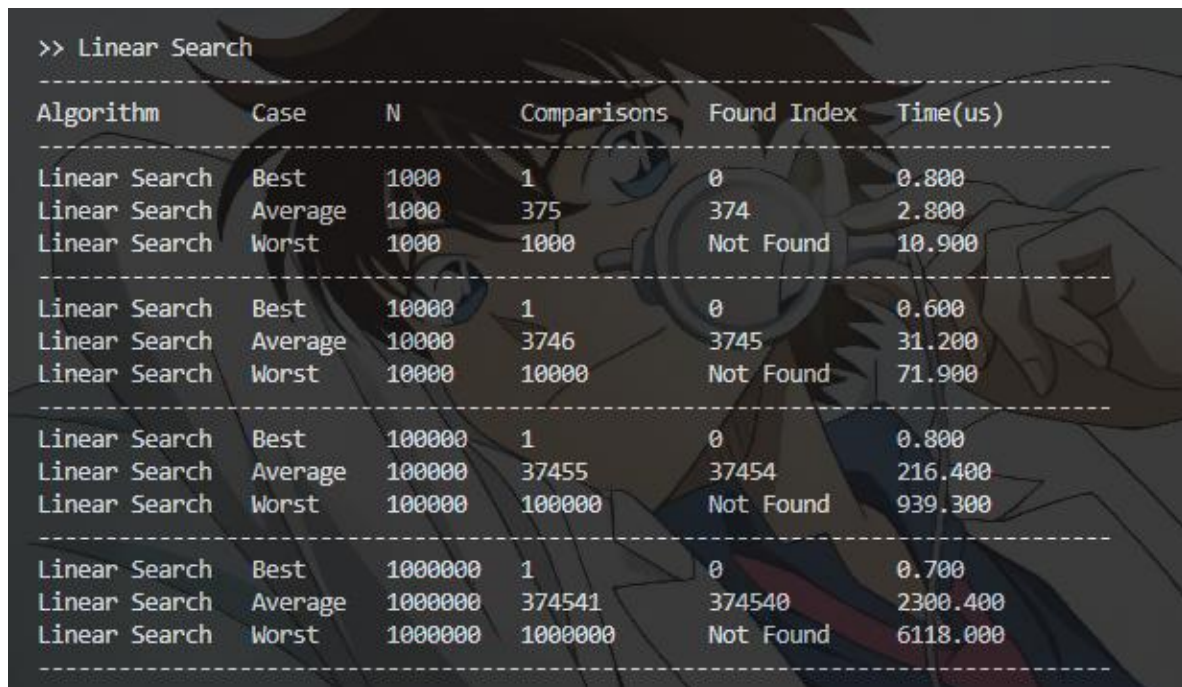
LINEAR SEARCH

A. Source Code

Tabel 1. Source Code Linear Search

1	<code>void linear_search(const std::vector<int>& data, int</code>
	<code>target, Metrics& m) {</code>
2	<code> auto start = Clock::now();</code>
3	
4	<code> m.comparisons = 0;</code>
5	<code> for (int i = 0; i < data.size(); i++) {</code>
6	<code> m.comparisons++;</code>
7	<code> if(data[i] == target) {</code>
8	<code> m.found_index = i;</code>
9	<code> break;</code>
10	<code> } else {</code>
11	<code> m.found_index = -1;</code>
12	<code> }</code>
13	<code> }</code>
14	<code> m.time_us = std::chrono::duration<double,</code>
	<code>std::micro>(Clock::now() - start).count();</code>
15	<code>}</code>

B. Output Program



```
>> Linear Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.800
Linear Search	Average	1000	375	374	2.800
Linear Search	Worst	1000	1000	Not Found	10.900
Linear Search	Best	10000	1	0	0.600
Linear Search	Average	10000	3746	3745	31.200
Linear Search	Worst	10000	10000	Not Found	71.900
Linear Search	Best	100000	1	0	0.800
Linear Search	Average	100000	37455	37454	216.400
Linear Search	Worst	100000	100000	Not Found	939.300
Linear Search	Best	1000000	1	0	0.700
Linear Search	Average	1000000	374541	374540	2300.400
Linear Search	Worst	1000000	1000000	Not Found	6118.000

Gambar 1. Screenshot Output Linear Search

C. Pembahasan

Ini adalah linear search, dengan pendekatan brute force. Cara kerjanya yaitu dengan for loop, untuk mencari target dan membandingkan semua data satu persatu dengan target. Kalau target ketemu, loop di break. Kalau target tidak ketemu, nilai m(found_index) jadi -1. Algoritma ini tidak memerlukan data terurut, karena semua data akan di-compare dengan target secara linear. Algoritma ini bersifat iteratif dan dapat diimplementasikan dengan rekursif, tapi kalau rekursif, algoritma ini akan memakan ruang yang banyak untuk call stack, hingga kompleksitas ruangnya $O(n)$. Selain itu, kalau dengan rekursif, bisa terjadi stack overflow jika data terlalu banyak.

Untuk best case, algoritma langsung menemukan target di elemen pertama, kompleksitas waktunya jadi $O(1)$, tidak perlu loop sampai N. Average case, algoritma melakukan loop dan compare sampai target, kompleksitas waktunya $O(n)$. Worst case, target tidak berada dalam data tersebut. Algoritma tetap melakukan loop sampai N sehingga kompleksitas waktunya $O(n)$. Untuk semua case, linear search langsung loop dan mengakses array yang ada, tidak membuat array baru, sehingga kompleksitas ruangnya $O(1)$.

Jumlah comparison tergantung pada letak target, karena ini brute force, semuanya di compare. Jika target berada di elemen pertama, cukup sekali compare. Semakin banyak jumlah comparison, semakin jauh letak elemen target. Nilai indeks found tergantung dimana letak elemen target. Kalau 0 berarti target ada di elemen pertama, kalau 374 berarti target berada di elemen 375, kalau -1 berarti target tidak ditemukan. Ukuran data mempengaruhi running time dan jumlah comparison jika bukan best case atau jika target bukan di elemen pertama. Semakin banyak data dan semakin jauh elemen target, jumlah comparison dan running time semakin tinggi.

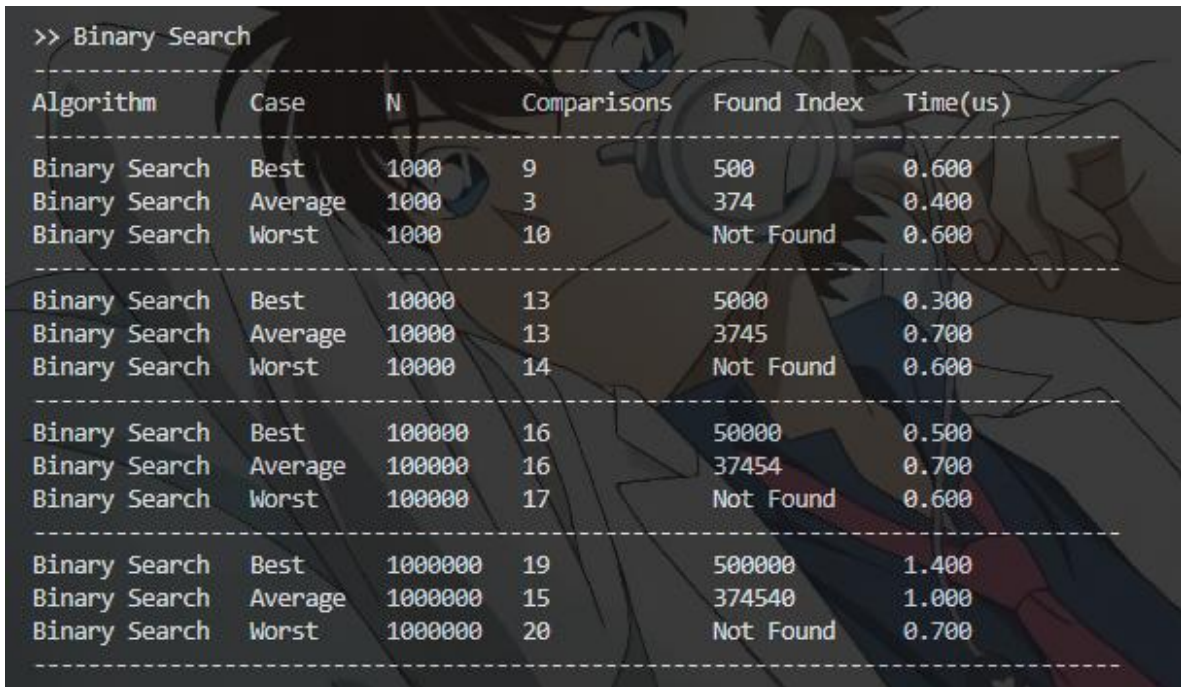
BINARY SEARCH

A. Source Code

Tabel 2. Source Code Binary Search

1	<code>void binary_search(const std::vector<int>& data, int</code>
	<code>target, Metrics& m) {</code>
2	<code> auto start = Clock::now();</code>
3	
4	<code> int low = 0;</code>
5	<code> int high = data.size() - 1;</code>
6	<code> m.comparisons = 0;</code>
7	
8	<code> while (low <= high) {</code>
9	<code> int mid = low + (high - low) / 2;</code>
10	
11	<code> if (data[mid] == target) {</code>
12	<code> m.found_index = mid;</code>
13	<code> break;</code>
14	<code> } else {</code>
15	<code> m.found_index = -1;</code>
16	<code> }</code>
17	<code> if (data[mid] < target) {</code>
18	<code> low = mid + 1;</code>
19	<code> } else {</code>
20	<code> high = mid - 1;</code>
21	<code> }</code>
22	<code> m.comparisons++;</code>
23	<code> }</code>
24	<code> m.time_us = std::chrono::duration<double,</code>
	<code>std::micro>(Clock::now() - start).count();</code>
25	<code>}</code>

B. Output Program



Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	0.600
Binary Search	Average	1000	3	374	0.400
Binary Search	Worst	1000	10	Not Found	0.600
Binary Search	Best	10000	13	5000	0.300
Binary Search	Average	10000	13	3745	0.700
Binary Search	Worst	10000	14	Not Found	0.600
Binary Search	Best	100000	16	50000	0.500
Binary Search	Average	100000	16	37454	0.700
Binary Search	Worst	100000	17	Not Found	0.600
Binary Search	Best	1000000	19	500000	1.400
Binary Search	Average	1000000	15	374540	1.000
Binary Search	Worst	1000000	20	Not Found	0.700

Gambar 2. Screenshot Output Binary Search

C. Pembahasan

Ini adalah binary search, metode serach dengan pendekatan divide and conquer. Cara kerjanya yaitu dengan while loop, selama nilai low tidak melebihi high. Algoritma langsung meloncat ke nilai mid, comparing nilai mid dengan target. Kalau nilai target lebih kecil dari mid, maka nilai high diubah ke $\text{mid} - 1$ agar algoritma mencari nilai mid lagi di sisi kiri mid sebelumnya. Kalau target lebih besar dari mid, nilai low diubah ke $\text{mid} + 1$ agar algoritma mencari nilai mid di sisi kanan mid sebelumnya. Search terus dilakukan hingga target ketemu tepat di mid dan loop di break. Kalau tidak ketemu, nilai m(found_indeks diisi -1. Algoritma ini memerlukan data terurut, karena agar algoritma tahu search dilanjutkan di sisi kiri mid (ketika target lebih kecil dari mid) atau di sisi kanan mid (ketika target lebih besar dari mid). Algoritma ini bersifat iteratif dan dapat diimplementasikan dengan rekursif, karena paradigmanya sama dengan merge sort, tetapi kalau memakai algoritma rekursif, kompleksitas ruangnya menjadi $O(\log n)$. Sehingga masih lebih bagus menggunakan algoritma iteratif (i guess).

Untuk best case, posisi target di tengah. Algoritma langsung menemukan datanya dalam 1 kali compare, karena elemen target persis di mid. Average case, posisi target bukan ditengah, algoritma akan mencari langsung ke tengah (mid), lalu jika target bukan di tengah, algoritma akan membuat nilai tengah lain, di sisi kanan mid sebelumnya atau di sisi kiri mid. Hal ini terus berulang hingga $mid == target$. Sehingga kompleksitas waktunya $O(n \log n)$, memiliki paradigma divide and conquer. Worst case, target tidak berada dalam data tersebut. Algoritma tetap menjalankan while loop hingga nilai low lebih tinggi dari high. Karena ini divide and conquer, kompleksitas waktunya tetap $O(n \log n)$. Untuk semua case, binary search langsung loop dan mengakses array yang ada, tidak membuat array baru, hanya menambah variabel low, mid, high. Sehingga kompleksitas ruangnya $O(1)$.

Jumlah comparison tergantung pada letak target, semakin dekat dengan mid atau low atau high, semakin tinggi jumlah comparison, karna ia membagi-bagi terus sampai ketemu pas di mid. Misalnya 51 dari 100, maka algoritma ke 50 dulu, lalu 75, lalu 62, lalu 56, 53, hingga akhirnya 51. Nilai indeks found tergantung dimana letak elemen target. Kalau 500 berarti target ada di elemen ke 501, kalau -1 berarti target tidak ditemukan. Ukuran data mempengaruhi running time dan jumlah comparison. Semakin banyak data dan semakin dekat elemen target dengan low/mid/high, jumlah comparison dan running time semakin tinggi.

ANALISIS PERFORMA

Perbandingan running time antar algoritma jelas lebih singkat binary search, untuk data 1 juta pun hanya memakan 0.7 sampai 1.4 microsecond. Sedangkan linear search jauh diatas itu, untuk best case hanya 0.3 microsecond, worst case mencapai 4178 microsecond. Binary search, dilihat dari running time untuk semua case yang tidak melebihi 2 microsecond. Sedangkan linear search apalagi worst case dengan data yang banyak, bisa mencapai 4 milisecond.

Algoritma linear search cocok untuk dataset kecil, selain itu juga tidak perlu data yang terurut karena brute force. Sedangkan binary search cocok untuk dataset besar, karena waktu yang diperlukan untuk searching jadi lebih singkat, namun data harus diurutkan dahulu.

Kondisi data yang terurut dan acak hanya memengaruhi linear search, jika elemen target itu kecil, maka waktu eksekusi makin singkat jika datanya sudah terurut. Namun jadi worst case jika targetnya 100 sedangkan jumlah datanya 100. Untuk binary search, datanya harus terurut, kalau tidak terurut, nilai mid akan salah dan target tidak akan ketemu.

TAUTAN GIT

Repository : <https://github.com/DSA26-ULM/module-5-searching-soraawistaria>